

Name: Anirudh G Achar

SRN: PES1UG24AM342

Subject: SE LAB 1

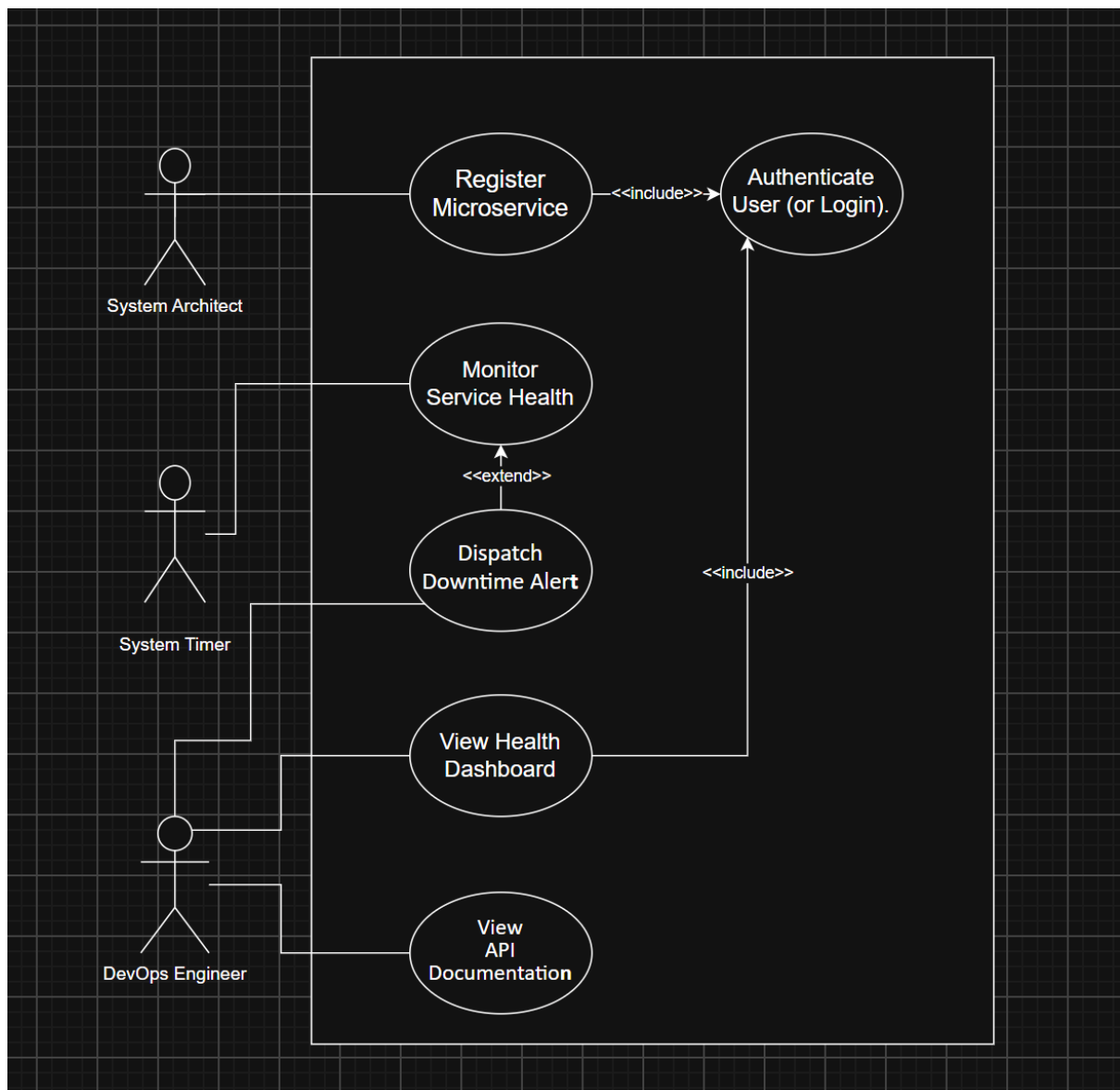
Topic: Internal Microservice Catalog & Health Portal

Requirement Table:

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale	Comments
FR-001	Functional	The system shall ping microservice health endpoints at 30-second intervals and compute rolling 24-hour service availability percentages.	High	Pass: Downtime recorded and alert dispatched on 3 consecutive failed health probes; Fail: Unreachable service marked healthy.	Core health monitoring loop.	
FR-002	Functional	The system shall aggregate and display API documentation for all registered microservices in a centralized portal.	Medium	Pass: Viewing a service page displays its associated API endpoints and payload schemas; Fail: Documentation is missing or fails to load.	Enables developers to easily discover API specs.	specify if it supports Swagger/OpenAPI
FR-003	Functional	The system shall allow a System Architect to manually register a new microservice by providing its endpoint URL and listing its dependencies.	High	Pass: New service appears in the catalog and dependency graph after submission; Fail: System rejects valid registration.	Required to build the dependency map.	Does system validate the endpoint URL format before accepting the registration?

FR-004	Functional	The system shall dispatch downtime alerts to the DevOps Engineer when a service fails its health check.	High	Pass: Email/SMS notification sent to DevOps engineer upon failure criteria met; Fail: No alert is sent.	Ensures quick response to system outages.	specify the exact notification channels
FR-005	Functional	The system shall display a unified dashboard showing the real-time status (Healthy/Down) and 24-hour availability percentage of all mapped services.	Medium	Pass: Dashboard loads all services with correct statuses and % metrics; Fail: Dashboard displays stale or incorrect data.	Provides a high-level operational overview.	What is default display if newly added service has < 24 hours of data?
NFR-001	Performance & Security	The service catalog dependency graph viewer must render interactive node architectures containing up to 200 services smoothly.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.	Ensures usability of the UI at enterprise scale.	
NFR-002	Performance	The system shall dispatch downtime alerts within 5 seconds of the 3rd consecutive failed health probe.	High	Pass: System logs show alert dispatch timestamp is < 5s after the final probe failure; Fail: Alert takes longer than 5s.	Immediate notification is critical for DevOps.	Make sure network latency is factored out of that 5-second measurement

UML Use-Case Diagram:



Use-Case Flow Document:

Use Case Name: Monitor Service Health

Primary Actor: System Timer

Secondary Actor: DevOps Engineer

Preconditions:

1. The target microservice is actively registered in the internal catalog.
2. The automated scheduling service is configured to run at 30-second intervals.

Main Success Scenario:

1. The System Timer initiates the scheduled health check process.
2. The system sends a ping request to the registered microservice health endpoint.
3. The microservice processes the request and responds with a successful status code.
4. The system logs the successful probe alongside the current timestamp.
5. The system computes the updated rolling 24-hour service availability percentage.
6. The system refreshes the internal health dashboard with the latest metrics.
7. The use case ends successfully.

Alternate Flow: Health Probe Failure

1. The system sends a ping request to the registered microservice health endpoint.
2. The microservice fails to respond within the timeout window or returns a server error.
3. The system logs the failed health probe.
4. The system evaluates the recent ping history for this specific microservice.
5. The system detects a threshold of 3 consecutive failed health probes.
6. The system triggers the extended use case to dispatch a downtime alert to the DevOps Engineer.
7. The system updates the dashboard to mark the service as unreachable and unhealthy.
8. The use case ends.

Postconditions:

1. The backend database accurately reflects the latest health status, probe history, and availability percentage for the queried microservice.